

DAY 2 Programs

NAME: AKASH J

Register No: 192324295

Program 1: First Palindromic String

Aim:

To find and return the first palindromic string from a given array of strings.

Algorithm:

1. Loop through each string word in the array words.
2. Check if the string reads the same forwards and backwards (i.e., `word == word[::-1]`).
3. If a match is found, immediately return the string.
4. If the loop completes without finding a palindrome, return an empty string "".

Code:

```
def firstPalindrome(words):  
    for word in words:  
        if word == word[::-1]:  
            return word  
    return ""
```

Input & Output:

• **Input 1:** words = ["abc", "car", "ada", "racecar", "cool"]

Output 1: "ada"

• **Input 2:** words = ["notapalindrome", "racecar"]

Output 2: "racecar"

Result:

The algorithm successfully finds the first palindromic string in $O(n \times m)$ time, where n is the number of words and m is the maximum length of a word.

Program 2: Elements in Both Arrays

Aim:

To calculate the number of indices in `nums1` that exist in `nums2`, and vice versa.

Algorithm:

1. Convert `nums1` and `nums2` into hash sets `set1` and `set2` to allow $O(1)$ lookups.
2. Initialize `answer1` and `answer2` to 0.
3. Iterate through `nums1`; if the element is in `set2`, increment `answer1`.
4. Iterate through `nums2`; if the element is in `set1`, increment `answer2`.
5. Return `[answer1, answer2]`.

Code:

```
def findIntersectionValues(nums1, nums2):  
    set1, set2 = set(nums1), set(nums2)  
    answer1 = sum(1 for num in nums1 if num in set2)  
    answer2 = sum(1 for num in nums2 if num in set1)  
    return [answer1, answer2]
```

Input & Output:

• **Input 1:** `nums1 = [2,3,2]`, `nums2 = [1,2]`

Output 1: `[2, 1]`

• **Input 2:** `nums1 = [4,3,2,3,1]`, `nums2 = [2,2,5,2,3,6]`

Output 2: `[3, 4]`

Result:

The program correctly counts the intersection frequencies in $O(n + m)$ time complexity.

Program 3: Sum of Squares of Distinct Counts

Aim:

To return the sum of the squares of distinct counts of all subarrays of a given array.

Algorithm:

1. Initialize `total_sum = 0`.
2. Use a nested loop to generate all possible subarrays. The outer loop `i` sets the start index.
3. Initialize an empty set `seen` for each starting index `i`.
4. The inner loop `j` expands the subarray from `i` to the end of the array.
5. Add `nums[j]` to `seen`. The distinct count is `len(seen)`.
6. Add the square of `len(seen)` to `total_sum`.

Code:

```
def sumCounts(nums):  
    total_sum = 0  
    n = len(nums)  
    for i in range(n):  
        seen = set()  
        for j in range(i, n):  
            seen.add(nums[j])  
            total_sum += len(seen) ** 2  
    return total_sum
```

Input & Output:

• **Input 1:** `nums = [1,2,1]`

Output 1: 15

• **Input 2:** `nums = [1,1]`

Output 2: 3

Result:

The sum of squared distinct counts is calculated successfully in $O(n^2)$ time.

Program 4: Divisible Pairs

Aim:

To count pairs (i, j) such that $\text{nums}[i] == \text{nums}[j]$ and $(i \times j)$ is divisible by k .

Algorithm:

1. Initialize $\text{count} = 0$.
2. Iterate through the array with two pointers, i from 0 to $n-1$, and j from $i+1$ to $n-1$.
3. Check if $\text{nums}[i] == \text{nums}[j]$ AND $(i * j) \% k == 0$.
4. If both conditions are met, increment count.
5. Return count.

Code:

```
def countPairs(nums, k):
    count = 0
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j] and (i * j) % k == 0:
                count += 1
    return count
```

Input & Output:

• **Input 1:** $\text{nums} = [3, 1, 2, 2, 2, 1, 3]$, $k = 2$

Output 1: 4

• **Input 2:** $\text{nums} = [1, 2, 3, 4]$, $k = 1$

Output 2: 0

Result:

The algorithm correctly identifies valid pairs in $O(n^2)$ time.

Program 5: Find Maximum Element

Aim:

To find the maximum element in an unsorted array using the most optimal time complexity.

Algorithm:

1. Set the maximum variable `max_val` to the first element of the array.
2. Iterate through the remaining elements.
3. If the current element is greater than `max_val`, update `max_val`.
4. Return `max_val`.

Code:

```
def findMax(arr):  
    if not arr: return None  
    max_val = arr[0]  
    for num in arr[1:]:  
        if num > max_val:  
            max_val = num  
    return max_val
```

Input & Output:

- **Input 1:** [1, 2, 3, 4, 5] → **Output 1:** 5
- **Input 2:** [7, 7, 7, 7, 7] → **Output 2:** 7
- **Input 3:** [-10, 2, 3, -4, 5] → **Output 3:** 5

Result:

The maximum value is found in $O(n)$ time and $O(1)$ space, which is the mathematical lower bound for this problem.

Program 6: Sort and Find Max Element

Aim:

To sort an array efficiently and then retrieve its maximum element from the sorted sequence.

Algorithm:

1. Handle the edge case: If the list is empty, return an appropriate message.
2. Sort the array using Python's built-in Timsort ($O(n \log n)$).
3. The maximum element will naturally reside at the last index (`arr[-1]`).

Code:

```
def sortAndFindMax(arr):  
    if not arr:  
        return "List is empty"  
    arr.sort()  
    return arr[-1]
```

Input & Output:

- Input 1: `[]` → Output 1: "List is empty"
- Input 2: `[5]` → Output 2: 5
- Input 3: `[3, 3, 3, 3, 3]` → Output 3: 3

Result:

The list is successfully sorted and the max value is retrieved.

Program 7: Unique Elements

Aim:

To create a new list containing only unique elements from the original list and analyze its space complexity.

Algorithm:

1. Initialize an empty list `unique_list` and an empty set `seen`.
2. Iterate through the input list.
3. If an item is not in `seen`, append it to `unique_list` and add it to `seen`.
4. Return `unique_list`.

Code:

```
def getUnique(arr):  
    seen = set()  
    unique_list = []  
    for num in arr:  
        if num not in seen:  
            unique_list.append(num)  
            seen.add(num)  
    return unique_list
```

Input & Output:

- **Input 1:** [3, 7, 3, 5, 2, 5, 9, 2] → **Output 1:** [3, 7, 5, 2, 9]
- **Input 2:** [-1, 2, -1, 3, 2, -2] → **Output 2:** [-1, 2, 3, -2]
- **Input 3:** [1000000, 999999, 1000000] → **Output 3:** [1000000, 999999]

Result:

The unique elements are successfully extracted using $O(n)$ time and $O(n)$ auxiliary space.

Program 8: Bubble Sort

Aim:

To sort an array of integers using the Bubble Sort technique and analyze its time complexity.

Algorithm:

1. Iterate through the array n times.
2. In each iteration, compare adjacent elements.
3. If they are in the wrong order ($arr[j] > arr[j+1]$), swap them.
4. Optimize by adding a boolean flag swapped. If no elements are swapped in the inner loop, the array is already sorted, and you can break early.

Code:

```
def bubbleSort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

Result:

The array is sorted in-place. Time Complexity is $O(n)$ (Best Case) and $O(n^2)$ (Worst/Average Case).

Program 9: Binary Search

Aim:

To check if a given number **x** exists in an array using Binary Search and analyze its time complexity.

Algorithm:

1. Sort the array (Binary Search requires a sorted array).
2. Initialize $low = 0$ and $high = len(arr) - 1$.
3. Loop while $low \leq high$.
4. Calculate $mid = (low + high) // 2$.
5. If $arr[mid] == key$, return the index mid .
6. If $arr[mid] < key$, adjust $low = mid + 1$.
7. If $arr[mid] > key$, adjust $high = mid - 1$.

Code:

```
def binarySearch(arr, key):
    arr.sort()
    low, high = 0, len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return f"Element {key} is found at position {mid}"
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return f"Element {key} is not found"
```

Input & Output:

- **Input 1:** $X=[3,4,6,-9,10,8,9,30]$, $KEY=10$
Output 1: Element 10 is found at position 5
- **Input 2:** $X=[3,4,6,-9,10,8,9,30]$, $KEY=100$
Output 2: Element 100 is not found

Result:

The element is successfully searched in **$O(\log n)$** time.

Program 10: $O(n \log n)$ Sort with Smallest Space

Aim:

To sort an array in ascending order without built-in functions in $O(n \log n)$ time complexity and $O(1)$ space complexity.

Algorithm (Heap Sort):

1. Build a Max-Heap from the input array.
2. Swap the root (maximum value) of the max-heap with the last element of the heap.
3. Reduce the heap size by 1 and heapify the root.
4. Repeat steps 2-3 until the heap is empty.

Code:

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[i] < arr[left]:
        largest = left

    if right < n and arr[largest] < arr[right]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def sortArray(nums):
    n = len(nums)

    for i in range(n // 2 - 1, -1, -1):
        heapify(nums, n, i)

    for i in range(n - 1, 0, -1):
        nums[i], nums[0] = nums[0], nums[i]
        heapify(nums, i, 0)

    return nums
```

Input & Output:

• **Input:** `nums = [5, 2, 3, 1]`

Output: `[1, 2, 3, 5]`

Result:

The array is sorted strictly meeting the $O(n \log n)$ time limit and $O(1)$ auxiliary space constraint.